

A scenic mountain landscape with green hills, rocky cliffs, and a cloudy sky. The foreground shows a rocky ridge with sparse vegetation. In the background, there are rolling green hills and a deep valley with a prominent rocky cliff face. The sky is filled with large, white, fluffy clouds.

AI & Software Engineering Workshop

Week 1, Day 4: Deploy

Edik Simonian, Summer 2026

Yesterday → today

Your bot has a persona, commands that **think**, and a **memory** that survives restarts.

One thing is still missing: it only runs **while your laptop is on**. Today it leaves your machine and goes **live on the internet**.

Production touches already in the repo

- **Rate limit:** `RATE_LIMIT` env var, default 250 messages/user/day
- **History trimming:** last 20 messages per user, 30-day expiry
- **Whitelist:** `ALLOWED_USERS` makes the bot **silent** to strangers
(silence, not rejection: scanners learn nothing)
- **Dedupe:** Telegram retries webhooks; the bot won't answer twice

All built on the same store you set up yesterday.

From polling to webhook

- All week: **polling**, your computer asks Telegram for updates
- Production: **webhook**, Telegram pushes to your server's URL
- PythonAnywhere runs the **same Flask app** as an always-on worker
- Same code, same `.env` idea: different delivery

Now your bot **stops needing your laptop**.

The front door: `api/index.py`

Route	Job
<code>/api/health</code>	"Is the worker alive?" → <code>OK</code>
<code>/api/webhook</code>	Where Telegram delivers messages
<code>/api/deploy</code>	Pulls new code on push (we'll wire this up)

- The webhook checks a **secret token**: forged requests get 403. The bot generates and registers it itself on first boot.
- `threaded=False` in `bot/clients.py`: handlers must finish in the request, or the platform may kill them mid-reply.

Deploy: one command

1. Sign up at pythonanywhere.com (free Beginner tier)
2. Create an API token: *Account* → *API token*
3. Add to your local `.env`:

```
PA_USERNAME=<your PA username>  
PA_API_TOKEN=<the token>
```

4. Ship it:

```
make deploy-pa
```

What just happened

The script created the web app, cloned **your fork**, built the venv, uploaded a production `.env`, including:

```
SQLITE_PATH=/home/<you>/bot.db  
WEBHOOK_URL=https://<you>.pythonanywhere.com/api/webhook
```

`WEBHOOK_URL` = auto-registration: every boot, the bot tells Telegram "*deliver here.*"
No manual `curl` needed.

Verify like an engineer

1. `https://<you>.pythonanywhere.com/api/health` → `OK`
2. Message your bot: the reply now comes from the server
3. **Quit the bot on your computer (`Ctrl+C`). Message it again.** 🎉
4. Something wrong? *Web tab* → *error log*: read the traceback

Push-to-deploy

One-time setup on your fork, in *Settings* → *Secrets* → *Actions*:

Secret	Value
DEPLOY_SECRET	random string (also in PA <code>.env</code>)
PA_DEPLOY_URL	<code>https://<you>.pythonanywhere.com/api/deploy</code>

Then:

```
git commit -am "New personality" && git push
```

~3 seconds later the live bot is updated. This is how real teams ship.

The fine print

- PA free tier: click the **"Run until 3 months from today"** renewal button **every 3 months**, or the app auto-disables
- PA emails you a week before. Set a phone reminder anyway.
- Free tier limits outbound internet to a whitelist: Telegram and Cerebras are on it

Today → Tomorrow

Today: your bot is **live on the internet**, updating itself on every push.

Tomorrow: the whole day is yours. You design and ship your **own** bot, start to finish.